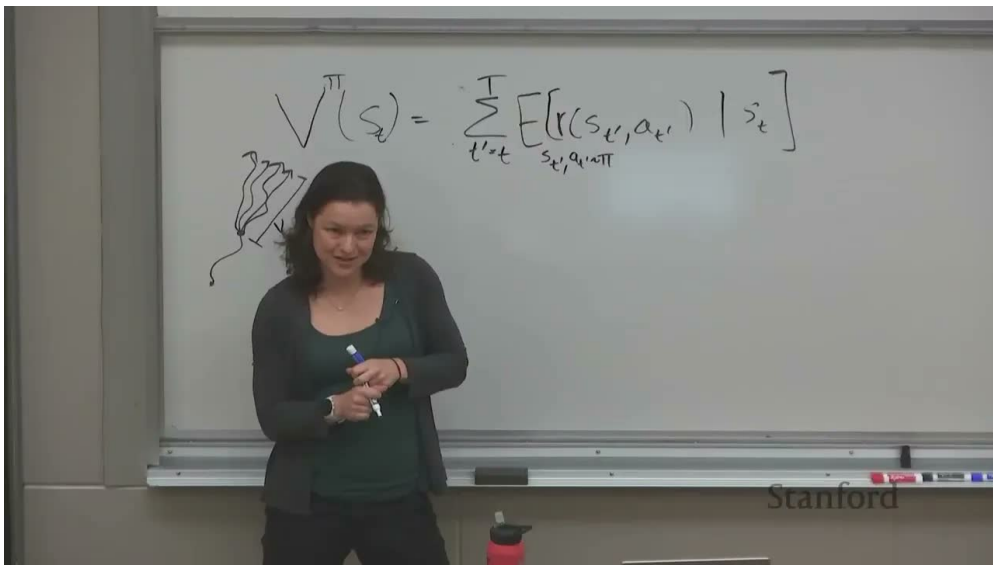


课程笔记

CS224R 第 4 讲: Actor-Critic Methods

五道口纳什 & Codex

2025-04-11



视频作者/频道: Stanford CS224R

发布日期: 2025-04-11

视频时长: 01:03:25

视频链接: <https://www.youtube.com/watch?v=oejFZShW9hU>

目录

1 问题动机：为什么 policy gradient 还不够好	2
1.1 三个基础对象：价值函数、Q 函数与 advantage	2
1.2 用鼓手例子理解 advantage	3
1.3 本章小结	3
2 如何估计价值：Monte Carlo、Bootstrap 与 N-step	3
2.1 Monte Carlo：直接拿回报当标签	3
2.2 Bootstrap：让模型帮助生成标签	4
2.3 N-step：在两者之间找中间地带	4
2.4 本章小结	4
3 Actor-Critic 全流程：从估值到策略更新	5
3.1 为什么 advantage 形式特别自然	5
3.2 本章小结	5
4 走向 off-policy：多步更新与 replay buffer	6
4.1 Version 1：同一批数据做多次梯度更新	6
4.2 Version 2：replay buffer 带来的真正 off-policy	6
4.3 本章小结	7
5 总结与延伸	7

1 问题动机：为什么 policy gradient 还不够好

本讲从上一讲的 vanilla policy gradient 出发。其核心更新写成

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right).$$

它的直觉很直接：未来回报高的动作要更常做，未来回报低的动作要更少做，基线 b 则负责把“高于平均”和“低于平均”区分开来。

但讲者强调，policy gradient 有两个结构性不足。第一，它完全 on-policy，每收一批数据通常只能做一次有效更新；第二，它直接把回报当监督信号，方差很大，学得慢、样本利用率低。于是本讲真正的目标变成：不要只看轨迹结果，而是先学会估计“一个状态有多好”“一个动作相对策略平均水平好多少”，再让策略沿着这些估计去改。

本讲主线

actor-critic 的关键不是多加一个网络这么简单，而是把 RL 中“评估好坏”和“根据好坏改策略”明确拆开。critic 负责估值，actor 负责更新，两者形成闭环。

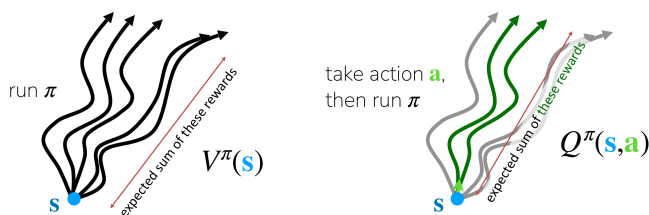
1.1 三个基础对象：价值函数、Q 函数与 advantage

为了搭起 actor-critic，讲者先回到三个最基础的对象。

Revisiting Some Useful Objects

value function $V^{\pi}(\mathbf{s})$ - future expected rewards starting at $\mathbf{s}$ and following π

Q-function $Q^{\pi}(\mathbf{s}, \mathbf{a})$ - future expected rewards starting at $\mathbf{s}$, taking $\mathbf{a}$, then following π



$$\text{Useful relation: } V^{\pi}(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot | \mathbf{s})} [Q^{\pi}(\mathbf{s}, \mathbf{a})]$$

advantage $A^{\pi}(\mathbf{s}, \mathbf{a})$ - how much better it is to take $\mathbf{a}$ than to follow policy π at state $\mathbf{s}$

$$A^{\pi}(\mathbf{s}, \mathbf{a}) = Q^{\pi}(\mathbf{s}, \mathbf{a}) - V^{\pi}(\mathbf{s})$$

图 1: 价值函数、Q 函数与 advantage 的关系示意

状态价值函数定义为

$$V^{\pi}(s) = \mathbb{E} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t = s, a_{t':T} \sim \pi \right].$$

它回答的问题是：如果我现在站在状态 s ，之后都老老实实按策略 π 行动，这个起点到底值多少钱。

动作价值函数则更细一层：

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t = s, a_t = a, a_{t+1:T} \sim \pi \right].$$

它不再只问“这个状态好不好”，而是问“在这个状态先强行采取动作 a ，再回到策略 π ，总收益会怎样”。

两者满足一个非常重要的关系：

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)].$$

也就是说， $V^\pi(s)$ 是按当前策略对所有动作的 Q 值做期望后的平均水平。

最后定义 advantage：

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

它测的不是绝对回报，而是“这个动作相对策略平均动作到底高出多少”。这正是 actor 更新时最想知道的量，因为策略并不需要知道绝对收益，只需要知道哪些动作比平均更值得强化。

1.2 用鼓手例子理解 advantage

讲者用“一个月内学会打鼓”的例子说明上述对象的区别。若一个人当前策略总是在沙滩休息，那么状态价值 $V^\pi(s)$ 很低；而在同一状态下，练鼓这个动作的 $Q^\pi(s, a)$ 会明显高于看电视或继续休息。于是练鼓对应的 advantage 为正，休息对应的 advantage 为负。

这个例子揭示了 actor-critic 的核心思想：策略更新不需要仅依赖最终成败，而可以用 advantage 直接判断“这一步到底是拉高了未来，还是拖累了未来”。

1.3 本章小结

本节完成了从“只看轨迹回报”到“显式刻画状态、动作与相对优势”的转变。后续所有 actor-critic 公式，本质上都在围绕 V^π 、 Q^π 与 A^π 三个对象展开。

2 如何估计价值：Monte Carlo、Bootstrap 与 N-step

critic 的存在价值在于提供更低方差、更可复用的监督信号。但一旦说“拟合价值函数”，新的问题立刻出现：标签从哪来？

2.1 Monte Carlo：直接拿回报当标签

最直接的方法是 Monte Carlo 估计。对于轨迹中的某个时刻 t ，直接把后续奖励和当作监督目标：

$$y_{i,t}^{\text{MC}} = \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}).$$

然后用监督学习拟合 $V_\phi^\pi(s_{i,t}) \approx y_{i,t}^{\text{MC}}$ 。

它的优点是无偏：不借助当前模型自己的预测，不会把错误再反馈给自己。缺点也很明显：必须等轨迹后半段都出来以后才能知道标签，而且方差很大，尤其在长时程任务中非常难学。

2.2 Bootstrap: 让模型帮助生成标签

另一条路线是 bootstrap, 也就是 temporal difference learning. 它用一步奖励加上下一状态的当前价值估计:

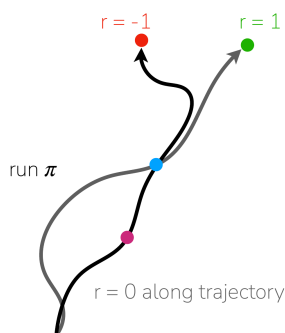
$$y_{i,t}^{\text{TD}} = r(s_{i,t}, a_{i,t}) + \gamma \hat{V}_{\phi}^{\pi}(s_{i,t+1}).$$

这样一来, 标签更快可得, 方差也更小, 但它引入了偏差, 因为标签的一部分来自一个还没学准的模型。

Estimating V^{π} : Monte Carlo vs. Bootstrap

Monte Carlo	$y_{i,t} = \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'})$	supervise with roll-out's summed rewards
Bootstrapped	$y_{i,t} = r(s_{i,t}, a_{i,t}) + \hat{V}_{\phi}^{\pi}(s_{i,t+1})$	supervise using reward and current value estimate

Let's look at an example:



Think-pair-share:

$$\begin{array}{ll} \hat{V}_{MC}^{\pi}(s) \approx ? & \hat{V}_{MC}^{\pi}(\textcolor{red}{s}) \approx ? \\ \hat{V}_{TD}^{\pi}(s) \approx ? & \hat{V}_{TD}^{\pi}(\textcolor{red}{s}) \approx ? \end{array}$$

Is there middle ground?
Can we balance bias and variance?

19

图 2: Monte Carlo 与 bootstrap 在偏差-方差上的权衡

图中的轨迹例子很典型。对靠近终点的状态, Monte Carlo 会直接看到最终 +1 或 -1 的结果; 而 bootstrap 则更依赖下一状态的估计值。因此 MC 更真实但更吵, TD 更稳定但可能系统性偏差。

2.3 N-step: 在两者之间找中间地带

为了兼顾两者优点, 本讲给出 N-step return:

$$y_{i,t}^{(n)} = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'}) + \gamma^n \hat{V}_{\phi}^{\pi}(s_{i,t+n}).$$

当 $n = 1$ 时它退化为 bootstrap; 当 n 足够大时则接近 Monte Carlo。实际做法往往是选择一个有限步长, 或使用加权和, 把不同 horizon 的信息综合起来。

为什么这很关键

critic 的学习质量, 直接决定 actor 看到的 advantage 是否可靠。估值标签若太噪, actor 会被高方差信号拖累; 估值标签若偏得太厉害, actor 会被系统性误导。

2.4 本章小结

估计价值函数并不是单一公式问题, 而是偏差和方差之间的工程取舍。Monte Carlo 更真实, bootstrap 更高效, N-step 则提供了折中方案; 这三者共同构成了本讲所谓的 policy evaluation。

3 Actor-Critic 全流程：从估值到策略更新

当 critic 可以产生价值估计之后，actor-critic 就能被写成一个完整循环。

A Full Algorithm Walkthrough

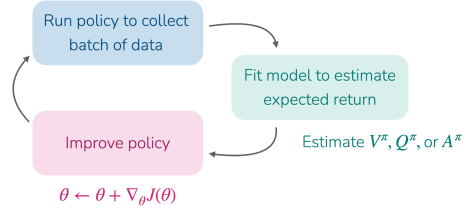
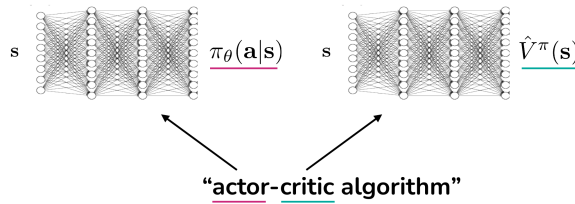
1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ

2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data

3. Evaluate $\hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) = r(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t,i}) \quad \forall t, i$

4. Evaluate $\nabla_\theta J(\theta) \approx \sum_{t,i} \nabla_\theta \log \pi_\theta(\mathbf{a}_{t,i} | \mathbf{s}_{t,i}) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$

5. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



$$y_{i,t} = \sum_{t'=t}^{t+n-1} r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{i,t+n})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

22

图 3: actor-critic 训练循环总览

讲者把流程拆成五步：

1. 用当前策略 π_θ 采样一批轨迹。
2. 用轨迹回报或 bootstrapped 目标拟合 V_ϕ^π 。
3. 用 critic 计算 advantage，例如

$$\hat{A}^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \hat{V}_\phi^\pi(s_{t+1}) - \hat{V}_\phi^\pi(s_t).$$

4. 用 advantage 构造策略梯度

$$\nabla_\theta J(\theta) \approx \sum_{t,i} \nabla_\theta \log \pi_\theta(a_{t,i} | s_{t,i}) \hat{A}^\pi(s_{t,i}, a_{t,i}).$$

5. 按梯度更新 actor。

相较于 vanilla policy gradient，这里最大的变化就是第三步。策略不再直接拿 reward-to-go 当权重，而是用 critic 先把“好坏”压缩为一个更干净的估计量。

3.1 为什么 advantage 形式特别自然

如果某动作的 Q^π 高于策略平均，即 $A^\pi(s, a) > 0$ ，那就应增大其概率；若低于平均，则应减小其概率。这样策略更新自动具有“以当前策略为参照系”的性质，比直接拿绝对回报更稳定。

3.2 本章小结

actor-critic 的完整闭环已经成型：数据收集、价值拟合、优势计算、策略更新。这里的 critic 不是附属品，而是把稀疏、高方差回报信号变成可训练目标的核心模块。

4 走向 off-policy: 多步更新与 replay buffer

前面虽然已经比 vanilla policy gradient 更高效, 但整体仍偏 on-policy。本讲后半段开始解决一个更现实的问题: 同一批数据能不能用很多次? 过去的的数据能不能继续用?

4.1 Version 1: 同一批数据做多次梯度更新

第一步扩展, 是在同一批由旧策略采到的数据上做多次更新。此时需要 importance ratio

$$\rho_t = \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)}$$

来校正 “数据来自旧策略、但梯度针对新策略” 的分布偏差。

Off-Policy Actor-Critic Methods

Version 1: Multiple Gradient Steps

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_{θ}
2. Fit $\hat{V}_{\phi}^{\pi_{\theta}}$ to summed rewards in data
3. Evaluate $\hat{A}^{\pi_{\theta}}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) = r(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_{\phi}^{\pi_{\theta}}(\mathbf{s}_{t+1,i}) - \hat{V}_{\phi}^{\pi_{\theta}}(\mathbf{s}_{t,i}) \quad \forall t, i$
4. Evaluate $\nabla_{\theta} J(\theta) \approx \sum_{t,i} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{t,i} | \mathbf{s}_{t,i}) \hat{A}^{\pi_{\theta}}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$ <- use importance weights here
5. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

25

图 4: 使用 importance weight 的多步 actor-critic 更新

这一步的意义在于样本复用, 但它也有明显限制: 如果新旧策略差异太大, ρ_t 会非常极端, 导致估计方差暴涨。因此它不是银弹, 只适合小步更新。

4.2 Version 2: replay buffer 带来的真正 off-policy

再往前一步, 就是 replay buffer。讲者把过去所有交互到的 transition 都放进缓冲区, 后续每次更新都从中抽样小批量数据。

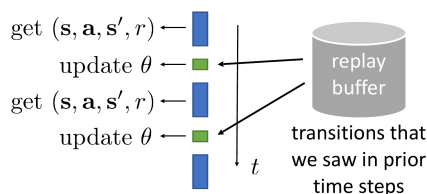
Off-Policy Actor-Critic Methods

Version 2: Replay buffers

actor-critic algorithm:

1. collect experience $\{s_i, a_i\}$ from $\pi_\theta(a|s)$ } Add this to replay buffer
2. fit $\hat{V}_\phi^\pi(s)$ to sampled reward sums
3. evaluate $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \gamma \hat{V}_\phi^\pi(s'_i) - \hat{V}_\phi^\pi(s_i)$ }
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i|s_i) \hat{A}^\pi(s_i, a_i)$ } Do this on minibatch sampled from all previous data
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

off-policy actor-critic



Slide adapted from Sergey Levine

29

图 5: replay buffer 让 actor-critic 从当前一批数据转向历史全部经验

这时问题也变得更尖锐：过去数据来自很多旧策略，直接拟合 $V^\pi(s)$ 会不一致，因为 V 默认把动作分布也包进去了。为了解决这个问题，后续课程会转向更适合 replay buffer 的 $Q(s, a)$ 视角，并进一步导向 SAC 等 fully off-policy 方法。

这里的风险

越 off-policy，样本效率通常越高；但算法也越依赖稳定的目标构造、合适的超参数以及更复杂的实现细节。数据效率和优化稳定性并不总是同向变化。

4.3 本章小结

从单次 on-policy 更新, 到 importance-weighted 多次更新, 再到 replay buffer, 本节展示了 actor-critic 如何逐步摆脱“每更新一次就得重新采样”的束缚。这为下一讲的 PPO、SAC 以及 value-based 方法铺平了道路。

5 总结与延伸

本讲完成了 online RL 里一个非常关键的结构升级。policy gradient 只知道“结果好不好”，而 actor-critic 试图回答“为什么好、哪一步好、在什么状态下更好”。这让 RL 从纯回报驱动，走向显式估值驱动。

从知识结构上看，本讲有三层收获。第一层是对象层：理解 V^π 、 Q^π 、 A^π 三者的关系。第二层是学习层：理解 Monte Carlo、bootstrap 与 N-step return 如何为 critic 提供标签。第三层是算法层：理解 actor-critic 如何把估值网络嵌入策略优化闭环，并进一步延伸到 off-policy 场景。

对后续课程而言，这一讲的影响非常大。PPO 本质上是“限制策略变化幅度的 actor-critic”，SAC 则是“借助 replay buffer 与 Q 学习的 fully off-policy actor-critic”。如果这一讲没吃透，后面看到 PPO/GAE/SAC 时就只会觉得公式越来越多；吃透以后会发现它们都只是围绕同一套估值逻辑。

辑做工程化强化。